Local-LLMs-Forensics

Reproduction Manual - Running the Experiments from Scratch

Prepared for the Local-LLM Forensics research dataset (TIFS submission)

September 2026

Contents

1 Introduction and Scope **2**
- 1.1 Purpose of this document . . . 2
- 1.2 Repository layout note . . . 2
- 1.3 Reproduction policy summary . . . 2

2 Experimental Findings Under Test **3**
- 2.1 Audited models . . . 3

3 System Requirements **4**
- 3.1 Hardware . . . 4
- 3.2 Software . . . 4
- 3.3 Environment verification . . . 4

4 Procedure **5**
- 4.1 Step 1 - Workspace construction . . . 5
- 4.2 Step 2 - Model acquisition and verification . . . 5
- 4.3 Step 3 - Runtime import . . . 5
- 4.4 Step 4 - Patched llama-server build . . . 5
- 4.5 Step 5 - Telemetry harness reconstruction . . . 6
- 4.6 Step 6 - Smoke test (approximately 10 minutes) . . . 6
- 4.7 Step 7 - Full audit (25-30 hours, unattended) . . . 6

5 Individual Experiment Recipes **8**
- 5.1 Canary injection (P1) . . . 8
- 5.2 Cross-tenant /slots exploit (P5) . . . 8
- 5.3 Patch overhead (P2) . . . 8
- 5.4 D2 and secure-allocator heap measurements . . . 8
- 5.5 LMS telemetry and adversarial phases . . . 8
- 5.6 Summary regeneration . . . 8

6 Troubleshooting **9**

7 Provenance and Ethical Disclosure **10**

1 Introduction and Scope

1.1 Purpose of this document

This manual specifies the complete procedure for reproducing, on a fresh Linux machine, every experiment contained in the Local-LLMs-Forensics dataset. It is the executable companion to the numbered scripts under `from-scratch/` (scripts 00-09), which implement each step described here.

The dataset supports the paper submission *"The Illusion of Local AI Security: Wrapper Forensics and Covert Reasoning Channels"* (IEEE TIFS, under review). Reproduction validates the dataset's published findings and permits independent verification of its claims.

1.2 Repository layout note

The public dataset places all artifacts under `Runs/`. The original research scripts, however, were written against a live workspace rooted at `~/research`, with model weights at `~/.lmstudio/models/`. Script `01_layout.sh` bridges this gap by constructing the expected workspace, either via symlinks into the repository clone (default) or by copying the files. The workspace root may be relocated by setting the `RESEARCH_ROOT` environment variable.

1.3 Reproduction policy summary

Item	Policy
Model weights	Auto-download (script 03), byte-pinned by SHA-256
llama.cpp patches	Rebuilt from shipped patch files (script 05)
Telemetry harness	Reconstructed (original not in dataset; script 06)
Full audit runtime	Approximately 25-30 hours, unattended
Failure handling	Non-fatal per phase; resumable via `PHASES_SKIP`

2 Experimental Findings Under Test

The full audit reproduces the following findings. Each row maps a published claim to its experiment and driver script.

Finding	Experiment	Driver
Canary leakage via visible content and hidden `reasoning_content`	5-template canonical injection, N=50 x 4 models	`n50_audit/run_injection_test.py`
Cross-tenant `/slots` session hijack (CWE-862 / CWE-200)	Tenant-A save -> tenant-B restore, N=50 x 4 models	`n50_audit/run_slot_exploit.py`
Security-patch latency overhead	Patched vs. unpatched, N=50 x 8 combinations	`n50_audit/run_kv_overhead.py`
Cross-model KV/heap residue (D4)	Sequential model loads with residue scan, 4 pairs x N	`d4_cross_model_residue.py`
PII attack chain (D7)	Synthetic PII prompt -> leak extraction, 4 models x N	`d7_pii_attack_chain.py`
GPU VRAM telemetry bypass	VRAM scan probes, N=50	`gpu_vram_v5.py`
Network-cleanliness control (D5)	netns `lmns` + tcpdump on `lmns-br`, 4 models x N	`telemetry_harness_ollama.py` (rebuilt)
Heap-string persistence (D2)	`multilayer_v2.py` x 3 + 12-tenant blast radius	`run_d2_measurements.sh`
Secure-allocator closure	D2 protocol with `SECURE_ALLOC=1`	`run_secure_allocator_test.sh`
LMS telemetry + adversarial phases	netns harness, 4 models; 8 attack vectors	`batch_4models.sh`, `lms-*-harness.py`

2.1 Audited models

All models are quantized Q4_K_M GGUF releases from `lmstudio-community`, byte-pinned by the SHA-256 digests in `Runs/provenance/model_digests.json` (also anchoring Zenodo DOI 10.5281/zenodo.20998459).

Short name	Hugging Face repository	Size
nemotron	`lmstudio-community/NVIDIA-Nemotron-3-Nano-4B-GGUF`	2.8 GB
qwen35	`lmstudio-community/Qwen3.5-9B-GGUF`	5.6 GB
gemma4	`lmstudio-community/gemma-4-E4B-it-GGUF`	5.3 GB
phi4	`lmstudio-community/Phi-4-reasoning-plus-GGUF`	9.0 GB

3 System Requirements

3.1 Hardware

Requirement	Minimum	Recommended
System RAM	16 GB (9B Q4_K_M, CPU inference)	32 GB
Free disk	40 GB	60 GB
CPU	x86_64, multi-core	-
Privileges	root/sudo (netns, iptables, tcpdump)	-

The disk budget covers the four GGUF files (~21 GB), the Ollama blob-store copies (~21 GB), and run artifacts.

3.2 Software

Linux (apt-based distribution assumed), Python 3 (standard library only; no pip packages are required), CMake and a C/C++ toolchain, `tcpdump`, `tshark`, `iproute2`, `iptables`, `tmux`, LM Studio (headless `lms` CLI), and Ollama.

3.3 Environment verification

```
bash from-scratch/00_check_env.sh       # audit present/missing components
bash from-scratch/02_install_prereqs.sh  # install missing system packages
```

4 Procedure

4.1 Step 1 - Workspace construction

```
bash from-scratch/01_layout.sh        # MODE=link (default) | MODE=copy
```

Constructs `~/research` (or `$RESEARCH_ROOT`) as the scripts expect:

```
~/research/
  scripts/            all harness drivers
  experiments/scripts/  canonical reproduction set
  experiments/build_index.py
  patches/            prompt_zeroize, kv_cache_zeroize
  provenance/         model SHA-256 digests
  logs/               run output (n50_audit/ within)
  .agents/skills/local-llm-telemetry-audit/scripts/   rebuilt in Step 5
  llama-cpp-build/    built in Step 4
```

4.2 Step 2 - Model acquisition and verification

```
bash from-scratch/03_download_models.sh            # asks once, then downloads (~21 GB)
bash from-scratch/03_download_models.sh --yes      # unattended / CI
bash from-scratch/03_download_models.sh --check    # verify only, never download
```

The script downloads each missing GGUF into `~/.lmstudio/models/lmstudio-community/<Repo>/` and verifies its SHA-256 against the provenance manifest. Files already present and verified are never re-downloaded. A digest mismatch triggers automatic deletion and one retry; a persistent mismatch must be reported upstream and must not be used.

4.3 Step 3 - Runtime import

```
bash from-scratch/04_import_models.sh
```

LM Studio. Models under `~/.lmstudio/models` are indexed automatically. Start the server headless (`lms server start`) so that `localhost:1234` answers and `lms load <id>` succeeds.

Ollama. The same GGUF files are imported *offline* via `ollama create <name>-local -f Modelfile` (`FROM <path>`). This is deliberate: the D5 network-cleanliness control must not touch the network. Ollama's blob store duplicates the files (~21 GB); when disk is constrained, import, test, and remove each model sequentially, as the original run did.

4.4 Step 4 - Patched llama-server build

```
bash from-scratch/05_build_llamacpp_patched.sh    # LLAMACPP_REF=master (override on drift)
```

The live build tree was never committed to the dataset. The script clones llama.cpp, applies `patches/prompt_zeroize.patch` and `patches/kv_cache_zeroize.patch`, writes `src/secure-allocator.cpp`, and builds via CMake.

The three security mechanisms under evaluation:

1. **prompt_zeroize.patch** - `explicit_bzero` on slot-owned plaintext strings (`generated_text`, the re-detokenized prompt copy, antiprompts, ...) at slot release (Sites A/B; `PROMPT_ZEROIZE_PATCH_REPORT.md`).
2. **kv_cache_zeroize.patch** - `zero_cell_data()` zeroes released K/V cache cells in both normal and transposed-V layouts (`KV_CACHE_PATCH_REPORT.md`).
3. **secure-allocator.cpp** - a global `operator delete` override that bzeros freed chunks >= 4 KB when `SECURE_ALLOC=1`, giving the patched/unpatched toggle that `run_kv_overhead.py` drives.

If a hunk fails to apply: upstream code has drifted. Pin `LLAMACPP_REF` to a commit near the patch dates (2026-06) and retry, or adapt offsets by hand; both patches are small and readable.

Scripts expect the binary at `llama-cpp-build/build/bin/llama-server`; if the build lands in `src/build/`, the script prints the symlink command.

4.5 Step 5 - Telemetry harness reconstruction

The D5/telemetry harness lived in a private skill directory and is not part of the public dataset. Script 06 reconstructs it from the run artifacts (`TELEMETRY_RUN_*/meta.json`) and the methodology in `TELEMETRY_FINDINGS_4MODELS.md` and `TELEMETRY_FINDINGS_OLLAMA.md`:

```
bash from-scratch/06_build_netns_harness.sh
bash from-scratch/07_build_ld_hook.sh     # net_hooks.so - L2 layer (LMS runs)
```

Protocol implemented per model:

1. `netns_isolate.sh up` - create namespace `lmns`, bridge `lmns-br`, veth pair, `10.99.0.0/24`, NAT via the host's default interface, `ip_forward=1`. Only the tool under test runs inside the namespace; co-resident host traffic cannot appear on `lmns-br` by construction.
2. Start `tcpdump` on `lmns-br` (L1, wire-level ground truth); for LM Studio runs additionally `LD_PRELOAD=net_hooks.so` (L2: `getaddrinfo` / `connect` tracing).
3. Model load, benign prompt, then a canary prompt (`TELEMETRY_CANARY_<uuid>`) inside the namespace.
4. Idle hold (3-4 minutes) to capture slow beacons.
5. Teardown; write `meta.json`, `pcap/capture.pcap`, `pcap_quick.json` (tshark-parsed flows, DNS, SNI, canary search), `triggers.jsonl`, `canary_report.md`.
6. Regenerate the summary tables: `python3 experiments/build_index.py`.

Ollama runs intentionally omit the L2 layer: Ollama is a Go binary, and libc-level hooking would silently miss Go's own syscalls - a documented false-negative risk.

4.6 Step 6 - Smoke test (approximately 10 minutes)

```
bash from-scratch/09_smoke_test.sh
```

Exercises every component with one minimal iteration: patched server boot, GGUF chat round-trip, `/slots` save/restore, injection script (n=2), one D4 run, netns up/connect/down, and a short Ollama harness pass. The full audit must only be started once this passes.

4.7 Step 7 - Full audit (25-30 hours, unattended)

```
tmux new -s n50audit
RESEARCH_ROOT="$HOME/research" bash from-scratch/08_run_all.sh
# detach: Ctrl-b d     reattach: tmux attach -t n50audit
```

Phases execute in fixed order and are individually resumable; progress is appended to `logs/n50_audit_master_<id>/progress`

#	Phase	Cost	Output
P1	`run_injection_test.py` (4 models x N)	60-90 min	`injection_ALL4_<id>.json`
P2	`run_kv_overhead.py` (8 combinations x N)	30-40 min	`kv_overhead_ALL_<id>.json`
P3	`d4_cross_model_residue.py` (4 pairs x N)	8-10 h	`d4_*_<id>.json`
P4	`d7_pii_attack_chain.py` (4 models x N)	4-6 h	`d7_*` runs
P5	`run_slot_exploit.py` (4 models x N)	~1 h	`slot_exploit_ALL4_<id>.json`
P6	`gpu_vram_v5.py` x N	1-2 h	VRAM probe runs

#	Phase	Cost	Output
P7	`telemetry_harness_ollama.py` (4 models x N)	15-20 h	`TELEMETRY_RUN_*_ollama_*`

Resuming after a crash. Inspect `progress.jsonl`, then skip completed phases by name:

```
PHASES_SKIP="injection_ALL4 kv_overhead_ALL" bash from-scratch/08_run_all.sh
```

Phase-name grammar: `d4_<a>_<b>_<i>`, `d7_<model>_<i>`, `gpuvram_<i>`, `d5_ollama_<model>_<i>`.

Safety design. The orchestrator runs with `set -uo pipefail` (a single phase failure never aborts the audit), imposes a four-hour ceiling per step, and cleans up (`pkill bin/llama-server`, `lms unload --all`) before and after every step. Kill patterns are deliberately scoped to the research binary path only.

5 Individual Experiment Recipes

5.1 Canary injection (P1)

```
python3 ~/research/scripts/n50_audit/run_injection_test.py \
    --model phi4 --iterations 50 --run-id debug1
```

Reports per-template leak rates (two explicit-ask templates, three benign). The explicit/benign split is what separates instruction-following from genuine disclosure.

5.2 Cross-tenant /slots exploit (P5)

```
python3 ~/research/scripts/n50_audit/run_slot_exploit.py \
    --model gemma4 --iterations 50 --run-id debug1
```

Success is defined as an exact `n_restored == n_saved` token-count match per iteration - the protocol-level proof used throughout the report. The script launches its own patched server on port 8094 with `--slot-save-path`.

5.3 Patch overhead (P2)

```
python3 ~/research/scripts/n50_audit/run_kv_overhead.py \
    --mode patched --model phi4 --iterations 50
```

5.4 D2 and secure-allocator heap measurements

```
bash ~/research/experiments/scripts/run_d2_measurements.sh        # patched x 3 + blast radius
bash ~/research/experiments/scripts/run_secure_allocator_test.sh  # SECURE_ALLOC=1 closure proof
```

Run these in an interactive terminal (they `pkill llama-server` between runs); each `multilayer_v2.py` run takes 2-5 minutes on Phi-4 (CPU).

5.5 LMS telemetry and adversarial phases

```
bash ~/research/experiments/scripts/batch_4models.sh     # 4-model netns telemetry
python3 ~/research/scripts/lms-adversarial-harness.py    # 8 adversarial vectors
python3 ~/research/scripts/lms-benign-harness.py         # 250 benign iterations (baseline)
```

If LM Studio runs on a Windows/WSL2 host, `lms_common.auto_detect_windows_host()` resolves it from `/etc/resolv.conf`; override with `LMS_HOST` / `LMS_PORT`.

5.6 Summary regeneration

```
cd ~/research && python3 experiments/build_index.py
```

6 Troubleshooting

Symptom	Cause / remedy
`run_injection_test.py` all-error	LM Studio server not running, or model IDs changed - `lms ls`, restart the server, check the `LMS_MODEL_ID` map
`/slots` endpoint returns 404	llama-server built without `--slot-save-path` support or too old - rebuild via script 05
Patches fail to apply	Upstream drift - pin `LLAMACPP_REF` near the patch date (2026-06) or adapt offsets manually
`iptables: Empty interface` during netns up	Host default interface unresolved - set it manually in `netns_isolate.sh`
Ollama `bind: address already in use`	A host `ollama serve` is still running - `systemctl stop ollama`; the harness starts its own instance inside the namespace
GGUF SHA-256 mismatch	Truncated download - script 03 deletes and retries automatically; a persistent mismatch must be reported upstream
Disk exhaustion during P7	Ollama blob copies - `ollama rm <name>-local` between models
D4/D5 slower than estimated	CPU-only inference dominates; `--n-gpu-layers 0` is intentional (matches the original measurements)

7 Provenance and Ethical Disclosure

1. **Model provenance.** Every published number is pinned to byte-identical weights by `Runs/provenance/model_digests.j` cross-anchored to Zenodo DOI 10.5281/zenodo.20998459.
2. **Responsible disclosure.** The cross-tenant `/slots` vulnerability (CWE-862 / CWE-200) was reported to the upstream maintainers through coordinated disclosure prior to public release; see `Runs/experiments/reports/DISCLosURE_PLAN.md`. The dataset ships no weaponized or mass-exploitation tooling.
3. **Experiment ethics.** All experiments are local, canary-token-based, and network-isolated by design.